



Parul[®]
University

NAAC **A++**
ACCREDITED UNIVERSITY

FACULTY OF ENGINEERING
AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

JAVA PROGRAMMING LABORATORY
(03010503PC04)

III SEMESTER

CSE Department



Laboratory Manual
Session 2026-27

CERTIFICATE

This is to certify that

Mr/ Ms _____ with enrollment no.

_____ has successfully completed
his/her laboratory experiments in the Java Programming Laboratory
(03010503PC04) from the department of COMPUTER SCIENCE &
ENGINEERING during the academic year 2026-2027.



Date of Submission:.....

Staff In charge:.....

Head Of Department:.....



TABLE OF CONTENT

Sr. No	Experiment Title	Date of Start	Date of Completion	Marks (out of 10)	Sign
1.	Write a Java program to display "Hello World" and demonstrate the Java program structure and demonstrate use of variables, data types, and type casting.				
2.	Write a Java program to perform arithmetic, relational, and logical operations, bitwise and other operators.				
3.	Write a Java program using below conditional statements. a) Even or Odd using if.. else statement. b) Roots of Quadratic Equation using else if ladder. c) Largest of three numbers using nested if else. d) Find out the week day using switch statement.				
4.	Write a Java program to demonstrate looping constructs: a) Reverse of a number using while loop. b) Prime number using do while loop. c) nth term of fibonacci sequence using for loop.				
5.	Write a Java program to perform methods and example on recursion.				
6.	Write a Java program to perform operations on one-dimensional and multi-dimensional arrays.				
7.	Write a Java program to demonstrate 1D- Arrays & 2-D Arrays. a) Maximum value and Second Maximum value without duplicates. b) Sort the names in Ascending Order. c) Addition of two matrix. d) 3x3 Matrix Multiplication				
8.	Write a Java program to demonstrate string handling using String, StringBuffer, and StringBuilder.				
9.	Write a Java program to check the word is palindrome or not.				
10.	Write a Java program to create a class and object and demonstrate constructors.				
11.	Write a Java program to demonstrate encapsulation using access specifiers.				



12.	Write a Java program to demonstrate inheritance: a) Single Inheritance. b) MultiLevel Inheritance. c) Hierarchical Inheritance. d) Hybrid Inheritance.				
13.	Write a Java program to demonstrate use of the this and super keywords				
14.	Write a Java program to demonstrate polymorphism (method overloading and overriding).				
15.	Write a Java program to implement abstraction using abstract classes and interfaces (Multiple Inheritance).				
16.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
17.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
18.	Write a Java program to implement List interface using ArrayList and LinkedList.				
19.	Write a Java program to implement Set interface using HashSet and TreeSet				
20.	Write a Java program to implement Queue interface using Priority Queue and Deque.				
21.	Write a Java program to implement map interface using HashMap and LinkedHashMap				
22.	Write a Java program to demonstrate multithreading using Thread class and Runnable interface				

PRACTICAL NO : 01

AIM:

Write a Java program to display “Hello World” and demonstrate the Java program structure and demonstrate the use of variables, data types, and type casting.

THEORY:

Java is a high-level, object-oriented programming language. Every Java program is written inside a class, and execution starts from the main() method.

Java Program Structure:

- **Class Declaration:** Define the class
- **Main Method:** Entry point of the program
- **Statements:** Instructions executed by the program

Variables:

Variables are named memory locations used to store data. Example: `int age = 19;`

Data Types:

- Int – integer values
- Float – decimal values
- Double – large decimal values
- Char – single characters
- Boolean – true or false
- String – sequence of characters

Type Casting:

Type casting is the process of converting one data type into another.

- **Implicit (Widening):** Automatically converts a smaller data type to a larger one.
- **Explicit (Narrowing):** Manually converts a larger data type to a smaller one using casting.

CODE SNIPPET:

```
class Practical1
{
    public static void main(String[]aa)
    {
        //Part1: Hello World

        System.out.println("Hello World!!");

        //Part2: Use of Variables & Data types

        double b = 20.0;

        int a = 20;

        byte c = 90;

        char d = 'Z';

        short e = 21;

        float f = 71.7f;

        boolean g = false;

        long h = 30000L;

        String i = "kshitij";

        System.out.println(b);

        System.out.println(a);

        System.out.println(c);

        System.out.println(d);

        System.out.println(e);

        System.out.println(f);

        System.out.println(g);

        System.out.println(h);

        System.out.println(i);
    }
}
```

```

//Part3: Type casting

//Implicit Type casting or widening

int z = 20;

double y = z;

System.out.println(y);

//Explicit Type Casting or Narrowing

double w = 69.0;

int v = (int) w;

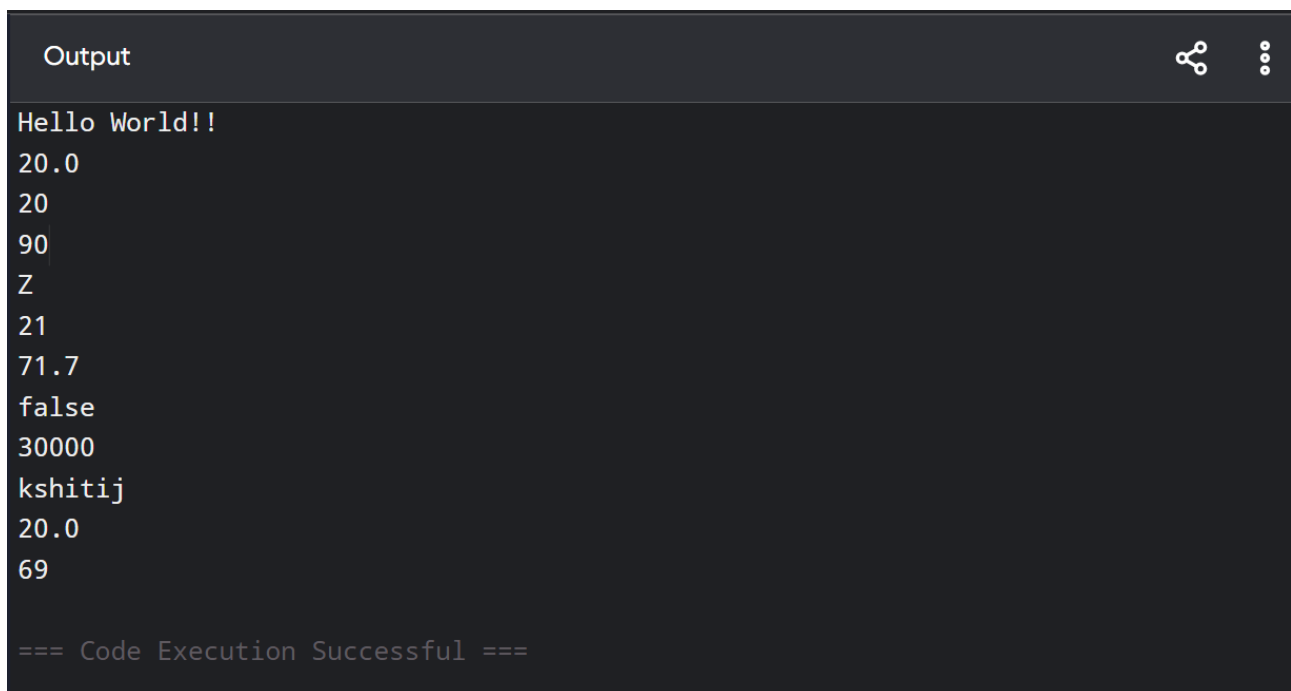
System.out.println(v);

}

}

```

OUTPUT:



The screenshot shows a dark-themed output window titled "Output". It contains the following text:

```

Hello World!!
20.0
20
90
Z
21
71.7
false
30000
kshitij
20.0
69

=== Code Execution Successful ===

```

CONCLUSION:

The Java program executed to display “Hello World.” The basic structure of a Java program was understood, along with the use of variables, data types, and type casting.

PRACTICAL NO : 02

AIM:

Write a Java program to perform arithmetic, relational and logical operations, bitwise and other operators.

THEORY:

Java provides several types of operators, including:

1. **Arithmetic Operators:** Used to perform mathematical operations such as addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).
2. **Relational Operators:** Used to compare two values. These include equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). The result of a relational operation is a Boolean value (true or false).
3. **Logical Operators:** Used to combine or negate Boolean expressions. The main logical operators are AND (&&), OR (||), and NOT (!).
4. **Bitwise Operators:** Used to perform operations directly on the individual bits of integer values. Java provides AND (&), OR (|), XOR (^), and complement (~) operators.
5. **Shift Operators:** Used to shift the bits of an integer value to the left or right. These include left shift (<<), signed right shift (>>), and unsigned right shift (>>>).
6. **Assignment Operators:** Used to assign values to variables. The basic assignment operator is (=), while compound operators include (+=), (-=), (*=), (/=), and (%=).
7. **Unary Operators:** Operate on a single operand. Examples include increment (++), decrement (--), unary plus (+), unary minus (-), and logical NOT (!).
8. **Conditional (Ternary) Operator:** The ternary operator (?:) is a shorthand way of writing a simple if-else expression. It evaluates a condition and returns one of two possible values.

CODE SNIPPET:

```
public class Practical2 {
    public static void main(String[] args) {
        int a = 20;
        int b = 10;

        // Arithmetic Operators
        System.out.println("Arithmetic Operators:");

        System.out.println("Addition      : " + (a + b));

        System.out.println("Subtraction   : " + (a - b));

        System.out.println("Multiplication : " + (a * b));

        System.out.println("Division      : " + (a / b));
    }
}
```

```

System.out.println("Modulus      : " + (a % b));

// Relational Operators

System.out.println("\nRelational Operators:");

System.out.println("a == b : " + (a == b));
System.out.println("a != b : " + (a != b));
System.out.println("a > b  : " + (a > b));
System.out.println("a < b  : " + (a < b));
System.out.println("a >= b : " + (a >= b));
System.out.println("a <= b : " + (a <= b));

// Logical Operators

boolean x = true;
boolean y = false;

System.out.println("\nLogical Operators:");
System.out.println("x && y : " + (x && y));
System.out.println("x || y : " + (x || y));
System.out.println("!x      : " + (!x));

// Bitwise Operators

System.out.println("\nBitwise Operators:");
System.out.println("a & b  : " + (a & b));
System.out.println("a | b  : " + (a | b));
System.out.println("a ^ b  : " + (a ^ b));
System.out.println("~a     : " + (~a));

// Shift Operators

System.out.println("\nShift Operators:");
System.out.println("a << 2 : " + (a << 2));
System.out.println("a >> 2 : " + (a >> 2));

```

```

System.out.println("a >>> 2: " + (a >>> 2));

// Unary Operators

System.out.println("\nUnary Operators:");

int n = 5;

System.out.println("++n : " + (++n));

System.out.println("--n : " + (--n));

System.out.println("-n : " + (-n));

// Assignment Operators

System.out.println("\nAssignment Operators:");

int c = 10;

c += 5;

System.out.println("c += 5 : " + c);

c -= 3;

System.out.println("c -= 3 : " + c);

c *= 2;

System.out.println("c *= 2 : " + c);

c /= 4;

System.out.println("c /= 4 : " + c);

// Ternary Operator

System.out.println("\nTernary Operator:");

int greater = (a > b) ? a : b;

System.out.println("Greater number: " + greater);

}

```

OUTPUT:

```
Output
Arithmetic Operators:
Addition      : 30
Subtraction   : 10
Multiplication : 200
Division      : 2
Modulus       : 0

Relational Operators:
a == b : false
a != b : true
a > b  : true
a < b  : false
a >= b : true
a <= b : false

Logical Operators:
x && y : false
x || y : true
!x     : false

Bitwise Operators:
a & b : 0
a | b : 30
a ^ b : 30
~a    : -21

Shift Operators:
a << 2 : 80
a >> 2 : 5
a >>> 2 : 5

Unary Operators:
++n : 6
--n : 5
-n  : -5

Assignment Operators:
c += 5 : 15
c -= 3 : 12
c *= 2 : 24
c /= 4 : 6

Ternary Operator:
Greater number: 20

=== Code Execution Successful ===
```

CONCLUSION:

The java program was successfully executed to demonstrate arithmetic, relational, logical, bitwise, assignment, unary, and ternary operators.

PRACTICAL NO : 03

AIM:

Write a Java program using below conditional statements.

- a) Even or Odd using if.. else statement.
- b) Roots of Quadratic Equation using else if ladder.
- c) Largest of three numbers using nested if else
- d) Find out the week day using switch statement.

THEORY:

(a) Even or Odd using if-else

The if-else statement is used to make decisions based on a condition. A number is checked using the modulus (%) operator. If the remainder when divided by 2 is zero, the number is even; otherwise, it is odd.

(b) Roots of Quadratic Equation using else-if Ladder

A quadratic equation is represented as $ax^2+bx+c=0$. The discriminant $D=b^2-4ac$ is used to determine the nature of its roots. An else-if ladder is used to handle the cases where the discriminant is positive, zero, or negative.

(c) Largest of Three Numbers using Nested if-else

Nested if-else means using one if-else statement inside another. It is used to compare three numbers by performing multiple comparisons and determining the largest value.

(d) Weekday using switch

The switch statement is used to select one option from multiple choices. In this program, numbers from 1 to 7 are used to represent the days of the week, and the corresponding weekday is displayed using different case statements.

CODE SNIPPET:

(a) Even or Odd using if-else

```
import java.util.Scanner;

public class EvenOdd {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");

        int n = sc.nextInt();

        if (n % 2 == 0) {

            System.out.println("Number is Even");

        } else {

            System.out.println("Number is Odd");

        }

    }

}
```

(b) Roots of Quadratic Equation using else-if Ladder

```
import java.util.Scanner;

public class QuadraticRoots { public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
    System.out.print("Enter a, b and c: ");
    double a = sc.nextDouble();
    double b = sc.nextDouble();
    double c = sc.nextDouble();

    double d = b * b - 4 * a * c;

    if (d > 0) {
        double r1 = (-b + Math.sqrt(d)) / (2 * a);
        double r2 = (-b - Math.sqrt(d)) / (2 * a);

        System.out.println("Two distinct real roots");
        System.out.println("Root 1 = " + r1);
        System.out.println("Root 2 = " + r2);
    } else if (d == 0) {
```

```

        double r = -b / (2 * a);

        System.out.println("Two equal real roots");
        System.out.println("Root = " + r);
    } else {
        System.out.println("Roots are complex");
    }
}
}

```

(c) Largest of Three Numbers using Nested if-else

```

import java.util.Scanner;

public class LargestThree {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter three numbers: ");

        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();

        int largest;

        if (a > b) {
            if (a > c) {
                largest = a;
            } else {
                largest = c;
            }
        } else {
            if (b > c) {
                largest = b;
            }
        }
    }
}

```

```

        } else {
            largest = c;
        }
    }
    System.out.println("Largest number = " + largest);
}
}

```

(d) Weekday using switch

```

import java.util.Scanner;

public class WeekDay {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter day number (1-7): ");
        int day = sc.nextInt();
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");

```

```
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day number");
    }
}
}
```

OUTPUT:(a) Even or Odd using if-else

Output

Enter a number: 4
Number is Even

=== Code Execution Successful ===

(b) Roots of Quadratic Equation using else-if Ladder

Output



```
Enter a, b and c: 1 5 6
Two distinct real roots
Root 1 = -2.0
Root 2 = -3.0
```

=== Code Execution Successful ===

(c) Largest of Three Numbers using Nested if-else

Output



```
Enter three numbers: 3 9 7
Largest number = 9
```

=== Code Execution Successful ===

(d) Weekday using switch

Output



```
Enter day number (1-7): 4
Thursday
```

=== Code Execution Successful ===

CONCLUSION:

The java program was successfully executed to demonstrate the use of if-else, else-if ladder, nested if-else, and switch statements for solving different problems.

PRACTICAL NO : 04

AIM:

Write a Java program to demonstrate looping constructs:

- a) Reverse of a number using while loop.
- b) Prime number using do while loop.
- c) nth term of fibonacci sequence using for loop.

THEORY:

(a) Reverse of a Number using while Loop

The while loop is used to repeatedly execute a block of statements as long as the given condition is true. To reverse a number, the last digit is obtained using the modulus (%) operator and added to the reversed number. The number is then divided by 10 repeatedly until it becomes zero.

(b) Prime Number using do-while Loop

A **prime number** is a number greater than 1 that has only two factors: 1 and itself. The do-while loop executes the statements at least once before checking the condition. In this program, it is used to check whether a number is divisible by any number other than 1 and itself.

(c) Nth Term of Fibonacci Sequence using for Loop

The **Fibonacci sequence** is a sequence in which each term is obtained by adding the previous two terms. It generally starts with 0, 1, 1, 2, 3, 5, The for loop is used to repeatedly calculate the next term until the required nth term is obtained.

CODE SNIPPET:

(a) Reverse of a Number using while Loop

```
import java.util.Scanner;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int reverse = 0;
        while (n != 0) {
            int digit = n % 10;
            reverse = reverse * 10 + digit;
            n = n / 10;
        }
        System.out.println("Reverse = " + reverse);
    }
}
```

(b) Prime Number using do-while Loop

```
import java.util.Scanner;

public class PrimeNumber {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");

        int n = sc.nextInt();

        int i = 2;

        boolean prime = true;

        if (n <= 1) {

            prime = false;

        } else {

            do {

                if (n % i == 0 && i != n) {

                    prime = false;

                    break;

                }

                i++;

            } while (i <= n / 2);

        }

        if (prime) {

            System.out.println("Number is Prime");

        } else {

            System.out.println("Number is Not Prime");

        }

    }

}
```

(c) Nth Term of Fibonacci Sequence using for Loop

```
import java.util.Scanner;
public class Fibonacci {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of n: ");
        int n = sc.nextInt();
        int a = 0, b = 1, c = 0;
        if (n == 1) {
            System.out.println("Nth term = " + a);
        } else if (n == 2) {
            System.out.println("Nth term = " + b);
        } else {
            for (int i = 3; i <= n; i++) {
                c = a + b;
                a = b;
                b = c;
            }
            System.out.println("Nth term = " + b);
        }
    }
}
```

OUTPUT:

(a) Reverse of a Number using while Loop

Output  

Enter a number: 9261
Reverse = 1629

=== Code Execution Successful ===

(b) Prime Number using do-while Loop

Output



Enter a number: 1920

Number is Not Prime

=== Code Execution Successful ===

(c) Nth Term of Fibonacci Sequence using for Loop

Output



Enter the value of n: 18

Nth term = 1597

=== Code Execution Successful ===

CONCLUSION:

The Java programs were successfully executed to demonstrate the use of while, do-while, and for loops. The programs helped in understanding looping constructs for reversing a number, checking prime numbers, and generating the Fibonacci sequence.

PRACTICAL NO : 05

AIM:

Write a Java program to perform methods and an example on recursion.

THEORY:

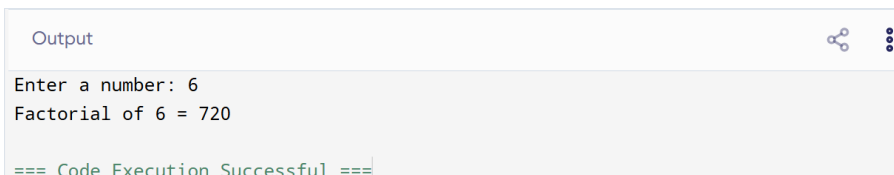
Method in Java is a block of code that performs a specific task. Methods help make a program organized, reusable, and easier to understand. A method can accept parameters and may return a value using the return statement.

Recursion is a technique in which a method calls itself repeatedly to solve a problem. A recursive method must contain a base condition, which stops the method from calling itself further. Recursion is commonly used for problems such as finding the factorial of a number, generating Fibonacci series, and calculating the sum of numbers.

CODE SNIPPET:

```
import java.util.Scanner;
class RecursionExample {
    static int factorial(int n) {
        if (n == 0 || n == 1) {
            return 1;
        }
        return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("Factorial of " + n + " = " + factorial(n));
        sc.close();
    }
}
```

OUTPUT:



The screenshot shows a terminal window titled "Output". It displays the prompt "Enter a number: 6" followed by the output "Factorial of 6 = 720". At the bottom, a green status bar indicates "=== Code Execution Successful ===".

CONCLUSION:

The Java program was successfully executed to demonstrate the use of methods and recursion

PRACTICAL NO : 06

AIM:

Write a Java program to perform operations on one-dimensional and multi-dimensional arrays.

THEORY:

A **Java array** is a collection of elements of the same data type stored under a single variable name. Arrays are used to store and process multiple values efficiently.

A **one-dimensional array** stores elements in a single row and is accessed using one index. Common operations include traversing, inserting, updating, searching, and finding the sum or average of elements.

A **multi-dimensional array** is an array containing other arrays. The most common example is a two-dimensional array, which represents data in rows and columns like a matrix. Elements are accessed using two indices: row and column.

CODE SNIPPET:

```
import java.util.Scanner;
class ArrayOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // One-dimensional array
        System.out.print("Enter the size of 1D array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println("\nElements of 1D array:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        // Sum of 1D array
        int sum = 0;
```

```

        for (int i = 0; i < n; i++) {
            sum += arr[i];
        }
        System.out.println("\nSum = " + sum);

// Multi-dimensional array
System.out.print("\nEnter rows and columns for 2D array: ");
int rows = sc.nextInt();
int cols = sc.nextInt();

int[][] matrix = new int[rows][cols];

System.out.println("Enter elements of 2D array:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = sc.nextInt();
    }
}

System.out.println("\nElements of 2D array:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}
}

```

OUTPUT:

Output



Enter the size of 1D array: 5

Enter 5 elements:

10 20 50 90 60

Elements of 1D array:

10 20 50 90 60

Sum = 230

Enter rows and columns for 2D array: 2 2

Enter elements of 2D array:

2 4 6 8

Elements of 2D array:

2 4

6 8

=== Code Execution Successful ===

CONCLUSION:

The Java program was successfully executed to demonstrate operations on one-dimensional and multi-dimensional arrays.

PRACTICAL NO : 07

AIM:

Write a Java program to demonstrate 1D - Arrays & 2-D Arrays.

- a) Maximum value and Second Maximum value without duplicates.
- b) Sort the names in Ascending Order.
- c) Addition of two matrix. d) 3x3 Matrix Multiplication

THEORY:

a) Maximum value and Second Maximum value without duplicates

A one-dimensional array is used to store multiple values of the same data type. In this program, the array is used to find the maximum and second maximum values without considering duplicate values.

The program compares each element with the current maximum and second maximum values. If the same value occurs more than once, it is ignored while finding the second maximum value.

b) Sort the names in Ascending Order

A one-dimensional array of strings can be used to store multiple names. The names can be arranged in ascending alphabetical order by comparing them with each other.

In Java, the `compareTo()` method is used to compare two strings alphabetically. The sorting process swaps the names when they are in the wrong order.

c) Addition of Two Matrices

A two-dimensional array is used to represent a matrix in Java. A matrix consists of rows and columns.

Matrix addition is possible when both matrices have the same number of rows and columns. The corresponding elements of the two matrices are added to obtain the resulting matrix.

d) 3×3 Matrix Multiplication

Matrix multiplication is performed using two-dimensional arrays. For multiplying two matrices, the number of columns in the first matrix must be equal to the number of rows in the second matrix.

For two 3×3 matrices, each element of the resultant matrix is calculated by multiplying the corresponding elements of a row of the first matrix with a column of the second matrix and adding the products.

CODE SNIPPET:

a) Maximum value and Second Maximum value without duplicates

```
import java.util.Scanner;

class MaxSecondMax {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the size of array: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int max = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;

        for (int i = 0; i < n; i++) {
            if (arr[i] > max) {
                secondMax = max;
                max = arr[i];
            } else if (arr[i] > secondMax && arr[i] != max) {
                secondMax = arr[i];
            }
        }

        System.out.println("Maximum value = " + max);

        if (secondMax == Integer.MIN_VALUE) {
            System.out.println("Second maximum value does not exist.");
        }
    }
}
```

```

    }
    else {
        System.out.println("Second maximum value = " + secondMax);
    }

    sc.close();
}
}

```

b) Sort the names in Ascending Order

```

import java.util.Scanner;

class SortNames {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of names: ");
        int n = sc.nextInt();
        sc.nextLine();

        String[] names = new String[n];

        System.out.println("Enter the names:");
        for (int i = 0; i < n; i++) {
            names[i] = sc.nextLine();
        }
    }
}

```

```

// Sorting names in ascending order
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (names[i].compareToIgnoreCase(names[j]) > 0) {
            String temp = names[i];
            names[i] = names[j];
            names[j] = temp;
        }
    }
}

System.out.println("\nNames in ascending order:");

for (int i = 0; i < n; i++) {
    System.out.println(names[i]);
}

sc.close();
}
}

```

c) Addition of Two Matrices

```

import java.util.Scanner;

class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```

```

System.out.print("Enter number of rows: ");

int rows = sc.nextInt();

System.out.print("Enter number of columns: ");

int cols = sc.nextInt();

int[][] A = new int[rows][cols];
int[][] B = new int[rows][cols];
int[][] C = new int[rows][cols];

System.out.println("Enter elements of first matrix:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        A[i][j] = sc.nextInt();
    }
}

System.out.println("Enter elements of second matrix:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        B[i][j] = sc.nextInt();
    }
}

// Matrix addition
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}

```

```

        }
    }

    System.out.println("\nResultant Matrix:");

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            System.out.print(C[i][j] + " ");
        }
        System.out.println();
    }

    sc.close();
}
}

```

d) 3 × 3 Matrix Multiplication

```

import java.util.Scanner;

class MatrixMultiplication {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] A = new int[3][3];
        int[][] B = new int[3][3];
        int[][] C = new int[3][3];
    }
}

```

```

System.out.println("Enter elements of first 3x3 matrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        A[i][j] = sc.nextInt();
    }
}

System.out.println("Enter elements of second 3x3 matrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        B[i][j] = sc.nextInt();
    }
}

// Matrix multiplication
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        C[i][j] = 0;

        for (int k = 0; k < 3; k++) {
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

System.out.println("\nResultant Matrix:");

for (int i = 0; i < 3; i++) {

```

```
        for (int j = 0; j < 3; j++) {  
            System.out.print(C[i][j] + " ");  
        }  
        System.out.println();  
    }  
    sc.close();  
}
```

OUTPUT:

a) Maximum value and Second Maximum value without duplicates

Output



```
Enter the size of array: 6  
Enter the elements:  
15 7 82 33 21 99  
Maximum value = 99  
Second maximum value = 82
```

=== Code Execution Successful ===

b) Sort the names in Ascending Order

Output



Enter the number of names: 6

Enter the names:

Shashi

Kshitij

Ashutosh

Sanjeev

Rounak

komal

Names in ascending order:

Ashutosh

komal

Kshitij

Rounak

Sanjeev

Shashi

=== Code Execution Successful ===

c) Addition of Two Matrices

Output



Enter number of rows: 2

Enter number of columns: 2

Enter elements of first matrix:

12 24

15 30

Enter elements of second matrix:

16 32

19 38

Resultant Matrix:

28 56

34 68

=== Code Execution Successful ===

d) 3×3 Matrix Multiplication

Output



Enter elements of first 3x3 matrix:

3 6 9

5 10 15

7 14 21

Enter elements of second 3x3 matrix:

2 4 6

4 8 12

6 12 18

Resultant Matrix:

84 168 252

140 280 420

196 392 588

=== Code Execution Successful ===

CONCLUSION:

The java programs were successfully executed to demonstrate the use of one-dimensional and two-dimensional arrays. We found the maximum and second maximum values, sorting names, performing matrix addition, and carrying out 3×3 matrix multiplication.

PRACTICAL NO : 08

AIM:

Write a Java program to demonstrate string handling using String, StringBuffer, and StringBuilder.

THEORY:

String handling in Java is used to create, store, and manipulate text. Java provides the String, StringBuffer, and StringBuilder classes for working with strings.

String class is immutable, which means its value cannot be changed after it is created. Whenever a modification is made, a new string object is created.

StringBuffer is mutable, so its contents can be changed without creating a new object. It is synchronized and is suitable when multiple threads may access the same string object.

StringBuilder is also mutable and provides similar operations to StringBuffer. However, it is not synchronized, so it is generally faster when thread safety is not required.

CODE SNIPPET:

```
class StringHandling {  
    public static void main(String[] args) {  
        // String  
        String str = "Hello";  
        System.out.println("String:");  
        System.out.println("Original String: " + str);  
        System.out.println("Length: " + str.length());  
        System.out.println("After concatenation: " + str.concat(" Java"));  
  
        // StringBuffer  
        StringBuffer sb = new StringBuffer("Hello");  
        System.out.println("\nStringBuffer:");  
        System.out.println("Original StringBuffer: " + sb);  
    }  
}
```

```
sb.append(" Java");  
System.out.println("After append: " + sb);  
  
sb.insert(6, "World ");  
System.out.println("After insert: " + sb);  
  
sb.reverse();  
System.out.println("After reverse: " + sb);  
  
// StringBuilder  
StringBuilder sbd = new StringBuilder("Hello");  
System.out.println("\nStringBuilder:");  
System.out.println("Original StringBuilder: " + sbd);  
  
sbd.append(" Java");  
System.out.println("After append: " + sbd);  
  
sbd.insert(6, "World ");  
System.out.println("After insert: " + sbd);  
  
sbd.reverse();  
System.out.println("After reverse: " + sbd);  
}  
}
```

OUTPUT:

```
Output  
```

```
String:
Original String: Hello
Length: 5
After concatenation: Hello Java

StringBuffer:
Original StringBuffer: Hello
After append: Hello Java
After insert: Hello World Java
After reverse: avaJ dlroW olleH

StringBuilder:
Original StringBuilder: Hello
After append: Hello Java
After insert: Hello World Java
After reverse: avaJ dlroW olleH

=== Code Execution Successful ===
```

CONCLUSION:

The Java program was successfully executed to demonstrate string handling using String, StringBuffer, and StringBuilder.

PRACTICAL NO : 09

AIM:

Write a Java program to check the word is palindrome or not.

THEORY:

A palindrome is a word, number, or sequence that remains the same when it is read from left to right or from right to left. Some examples of palindrome words are madam, level, radar, and civic.

In Java, a palindrome word can be checked using string handling methods. The program first accepts a word from the user and then reverses the word character by character using a for loop. The `charAt()` method is used to access each character of the string from the last position to the first position. The reversed word is then compared with the original word using the `equalsIgnoreCase()` method.

If the original word and the reversed word are the same, the word is considered a palindrome. Otherwise, the word is not a palindrome. This program demonstrates the use of strings, loops, `length()`, `charAt()`, and string comparison in Java.

CODE SNIPPET:

```
import java.util.Scanner;
class Palindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String word = sc.nextLine();

        String reverse = "";

        // Reverse the word
        for (int i = word.length() - 1; i >= 0; i--) {
            reverse = reverse + word.charAt(i);
        }

        // Check palindrome
        if (word.equalsIgnoreCase(reverse)) {
            System.out.println(word + " is a palindrome.");
        } else {
```

```
        System.out.println(word + " is not a palindrome.");
    }

    sc.close();
}
}
```

OUTPUT:

Output  

Enter a word: MalayalaM
MalayalaM is a palindrome.

=== Code Execution Successful ===

CONCLUSION:

The Java program was successfully executed to check whether a given word is a palindrome or not.

PRACTICAL NO : 10

AIM:

Write a Java program to create a class and object and demonstrate constructors.

THEORY:

A class in Java is a blueprint used to create objects. An object is an instance of a class and is created using the new keyword.

A constructor is a special method that is automatically called when an object is created. It has the same name as the class and does not have a return type. Constructors are used to initialize the data members of an object.

Java supports different types of constructors, such as default and parameterized constructors. A default constructor does not take any arguments, while a parameterized constructor accepts values to initialize an object.

CODE SNIPPET:

```
class Student {  
  
    int rollNo;  
    String name;  
    int age;  
  
    // Default constructor  
    Student() {  
        rollNo = 0;  
        name = "Unknown";  
        age = 0;  
    }  
    // Parameterized constructor  
    Student(int r, String n, int a) {  
        rollNo = r;  
        name = n;  
        age = a;  
    }  
}
```

```
}

void display() {
    System.out.println("Roll No: " + rollNo);
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
}
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student(101, "Rahul", 20);

        System.out.println("Student 1:");
        s1.display();

        System.out.println("\nStudent 2:");
        s2.display();
    }
}
```

OUTPUT:

```
Output  
```

```
Student 1:  
Roll No: 0  
Name: Unknown  
Age: 0  
  
Student 2:  
Roll No: 101  
Name: Rahul  
Age: 20  
  
=== Code Execution Successful ===
```

CONCLUSION:

The Java program was successfully executed to create a class and objects and demonstrate the use of constructors.

PRACTICAL NO : 11

AIM:

Write a Java program to demonstrate encapsulation using access specifiers.

THEORY:

Encapsulation is an important concept in Java that combines data and methods into a single class. It protects the data from direct access by making variables private and providing public methods to access and modify them.

Java provides access specifiers such as private, default, protected, and public to control the visibility of class members. In encapsulation, private variables are commonly accessed using public getter and setter methods.

CODE SNIPPET:

```
class Student {  
  
    // Private data members  
    private int rollNo;  
    private String name;  
  
    // Public setter methods  
    public void setRollNo(int r) {  
        rollNo = r;  
    }  
  
    public void setName(String n) {  
        name = n;  
    }  
  
    // Public getter methods  
    public int getRollNo() {  
        return rollNo;  
    }  
  
    public String getName() {  
        return name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
Student s = new Student();

// Setting values using setter methods
s.setRollNo(101);
s.setName("kshitij");

// Getting values using getter methods
System.out.println("Roll No: " + s.getRollNo());
System.out.println("Name: " + s.getName());
    }
}
```

OUTPUT:

Output  

Roll No: 101
Name: kshitij

=== Code Execution Successful ===

CONCLUSION:

The Java program was successfully executed to demonstrate encapsulation using access specifiers.